Regulárne výrazy

Vyhľadávanie informácií

Martin Šeleng

All info

http://regex.info/

http://www.regular-expressions.info/tutorial.html

Real Problems

- ^(From|Subject):
- Parsing not valid XML (nie je už také aktuálne)
- Replacing text in multiple files
  - **sed -i 's/200[0-9]\{7\}/2005102901/' ./***
- Extracting URL
  - **(.+)**
- Crawler obmedzenia
  - **.+\.stuba\.sk**
  - **.*sav(ba)?\.sk**

Special characters - metacharacters

- ^cat$, ^$, ^
  - ^$ nematchuju žiadny znak iba pozíciu
- gr[ea]y
- egrep ‘q[^u]’ word.list
  - not match Qantas, Iraq
  - Iraqi
  - Iraquian
  - miqra
  - qasida
  - zaqqum

Special characters

- 03.19.76 better 03[-./]19[-./]76
  - **Lottery #: 19 203319 7639**
  - **Email problem v.i.a.g.r.a**
- gray|grey, gr(a|e)y, **gr[ae]y only one char**
  - **Wrong gr[a|e]y, gra|ey**
  - **(First|1st) [Ss]treet**
  - **(Fir|1)st [Ss]treet**
  - **^From|Subject|Date: ^(From|Subject|Date):**
  - **[fF][rR][oO][mM]**
  - **egrep –i '^(From|Subject|Date):' mailbox**

Special characters

- egrep
  - \<cat\> word boundary if implemented
- [^x]
  - Hocičo okrem x (aj prázdny riadok)
  - Niečo čo nie je x (niečo tam musí byť)
- colou?r
  - color, colour, semicolon
- July 4th , Jul 4
  - (July|Jul)
  - July? 4(th)?

Platnosť

- From|Subject – celý string po zátvorky
- iba jeden znak alebo v zátvorkách
  - **Colou?r**
  - **<h[1-6] *>**
  - **<hr +size *= *[0-9]+ *>**
- <hr(+size *= *[0-9]+)? *>
- 0[xX][a-fA-F0-9] – hexa decimálne

Backreference and dot

- Chcem nájsť rovnaké slová (e.g. the the)
  - **the the**
  - **the +the**
  - **([a-z]+) +\1**
    - \1 \2 \3 podľa zátvoriek (opakovanie reg. výrazu)
- Dot (.)
  - **ega.att.com**
    - matchne aj “megawatt computing”
    - Oprava – ega\.att\.com
  - **\([a-z]+\)**
    - matchne “(very)”

? *

- Does not have to match anything
- 10,05 SK
  - **([0-9]+(,[0-9]+)?) – match 10**
  - **([0-9]+(,[0-9]+)?) *(Sk|SKK) match 10,05 Sk**
- URL
  - **\<http://[^]+\.html?\>**
    - Not very good but can be enough

Čas, Summary

- Anglický
  - **9:17 am, 12:30 pm**
  - **1?[0-9] allows 19 aj 9**
  - **(1[012]|[1-9]):[0-5][0-9] (am|pm)**
- Slovenský 24 hod aj s počiatočnou nulou
  - **([01]?[0-9]|2[0-3]):[0-5][0-9]**
  - **([012]?[0-3]|[01]?[4-9])**
- Summary – strana 32 - regex.info

Objekty Príklady

- Ontea patterns XML
  - http://ontea.cvs.sourceforge.net/viewvc/ontea/OnteaSF/dist/patterns.xml?view=markup
- Ontea patterns properties
  - http://ontea.cvs.sourceforge.net/viewvc/ontea/OnteaSF/dist/patterns.properties?revision=1.1&view=markup

Java

```
String patternStr = "b";
Pattern pattern = Pattern.compile(patternStr);
//Determine if pattern exists in input CharSequence
inputStr = "a b c b";
Matcher matcher = pattern.matcher(inputStr);
boolean matchFound = matcher.find(); // true
//Get matching string
String match = matcher.group(); // b
//Get indices of matching string
int start = matcher.start(); // 2
int end = matcher.end(); // 3
//The end is index of the last matching character+1
//Find the next occurrence
matchFound = matcher.find();
```

Find

```
Pattern p = Pattern.compile(pattern);
Matcher m = p.matcher(text);
while(m.find()) {
   String foundString = null;
   String foundStringFull = m.group().trim();
   if (m.groupCount() == 0) {
     foundString = m.group().trim();
   } else {
     foundString = m.group(1).trim();
   }
}
```

Replace

```
Pattern p = Pattern.compile("[^A-Za-z0-9]");
Matcher m = p.matcher(name);
StringBuffer sb = new StringBuffer();
while (m.find()) {
   m.appendReplacement(sb, "_");
}
m.appendTail(sb);
name = sb.toString();
```

Unicode

```
Pattern p = Pattern.compile(pattern,
  Pattern.UNICODE_CASE);
```

java.util.regex.Pattern

- \p{Lu} – upercase
- \p{L} – all
  - **Unicode**
- \b
  - **Word boundaries**
  - **\bis\b**
    - This **is** island
  - **Treba písať \\b \\. (v Jave**
- \b(\w+)\s+\1\b
  - **Hocaké 2 rovnaké slová po sebe**

Positive and Negative Lookahead

- Negative lookahead
  - **q(?!u)**
    - Nájdi q, ktoré nie je nasledované u
- Difference with q[^u]
  - **Nasledujúci znak nie je súčasťou výsledku**
- Positive lookahead
  - **q(?=u)**
    - Nájdi q nasledované u

Positive and Negative Lookbehind

- Negative Lookbehind
  - **(?<!a)b**
    - Nájdi b pred ktorým nie je a
- Positive Lookbehind
  - **(?<=a)b**
    - Nájdi b pred ktorým je a

Emulating “near” with a Regular Expression

- \bword1\W+(\w+\W+){1,6}word2\b
  - \bword1\W+(?:\w+\W+){1,6}word2\b
    - Matchne všetko ako jeden výsledok
    - ?:
      - non-capturing group – nechytí to ako ďalšiu grupu
- \bword1\W+(\w+\W+){1,6}word2\b|\bword2\W+(\w+\W+){1,6}word1\b

Summary

Výrazy ktoré reprezentujú hľadanie jedného znaku		
.	Bodka	zodpovedá ľubovoľnému znaku
[ ]	Množina	zodpovedá uvedenému znaku z množiny
[^]	Negatívna množina	zodpovedá znaku ktorý nie je uvedený
\znak	Znak s lomítkom	použijeme na hľadanie špeciálnych znakov pre regexy ako aj definíciu ďalších množín alebo pozícií
Výrazy ktoré sa pridávajú na počítanie znakov, kvantifikátory		
?	Otáznik	nula alebo jeden znak
*	Hviezdička	ľubovoľný počet znakov (aj nula)
+	Plus	minimálne jeden znak alebo viac
{*min*,*max*}	Rozsah	rozsah od *min* po *max*
Výrazy ktoré reprezentujú hľadanie pozície		
^	Strieška	začiatok riadku alebo textu (podľa nastavenia)
$	Dolár	koniec riadku alebo textu (podľa nastavenia)
\b	Hranica slova	hranica slova v jazyku Python/Java. V egrep sa používa \<, \>
Výrazy ktoré reprezentujú hľadanie pozície		
|	Alebo	zodpovedá nájdeniu ktoréhokoľvek výrazu oddeleného pomocou |
( )	Zátvorky	ohraničuje rozsah alebo slúži na určenie skupiny pre kvantifikátory a definíciu skupiny pre referencie
\1, \2, ...	Referencie	zodpovedá textu v prvej, druhej a ďalších zátvorkách

PHP

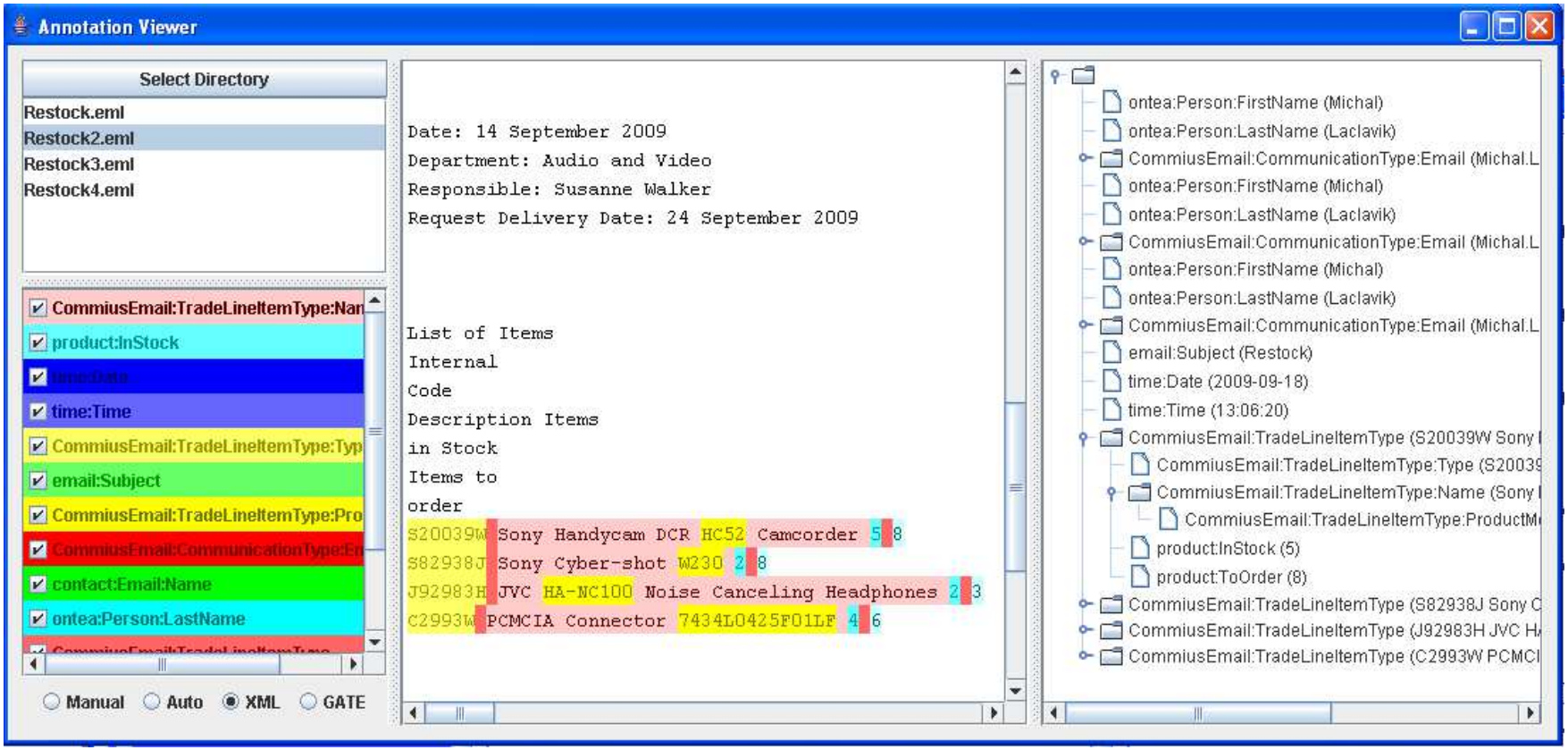
```
function node($xml, $deliminer) {
        if (ereg("<$deliminer>(.*)</$deliminer>",$xml, $out))
            return $out[1];
        else
            return "";
}
```

Support

- Perl m/regex/, r/regex/
- PHP eger, egeri, ereg_replace, \\
- Java form 1.4
  - \\
- Dot.net
- Python
- …

Ontea

- Založené na regulárnych výrazoch
- Podporuje zložené a vnorené patterny
- GUI na vizualizáciu výsledkov
- Podpora konverzie eml (emailov) do txt



Ontea Extraction Model

- Extraction based on JAVA Regular Expressions
- Model supports:
  - named backreferences
  - macros
- Result of extraction is set of **Key=>Value** pairs
- Key=>Value pairs (**Results**) are further processed

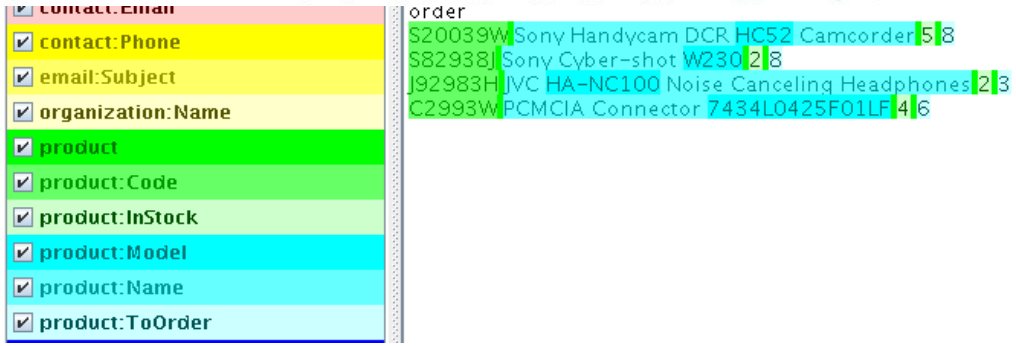
```
<!-- Product code -->

<pattern name="PRODUCT_CODE" class="product:Code">
    <regexp><![CDATA[[A-Z][0-9]{4,5}[A-Z]]]></regexp>
</pattern>

<pattern name="PRODUCT_MODEL" class="product:Model">
    <regexp><![CDATA[[-_A-Z0-9]+]]></regexp>
</pattern>

<pattern>
    <regexp><![CDATA[(\p{pattern:PRODUCT_CODE} *([^\n]* +\p{pattern:PRODUCT_MODEL} +[^\n]*) *([0-9]) ([0-9])) *\n]]></regexp>
    <groups>
        <group class="product">1</group>
        <group class="product:Description">2</group>
        <group class="product:InStock">3</group>
        <group class="product:ToOrder">4</group>
    </groups>
</pattern>

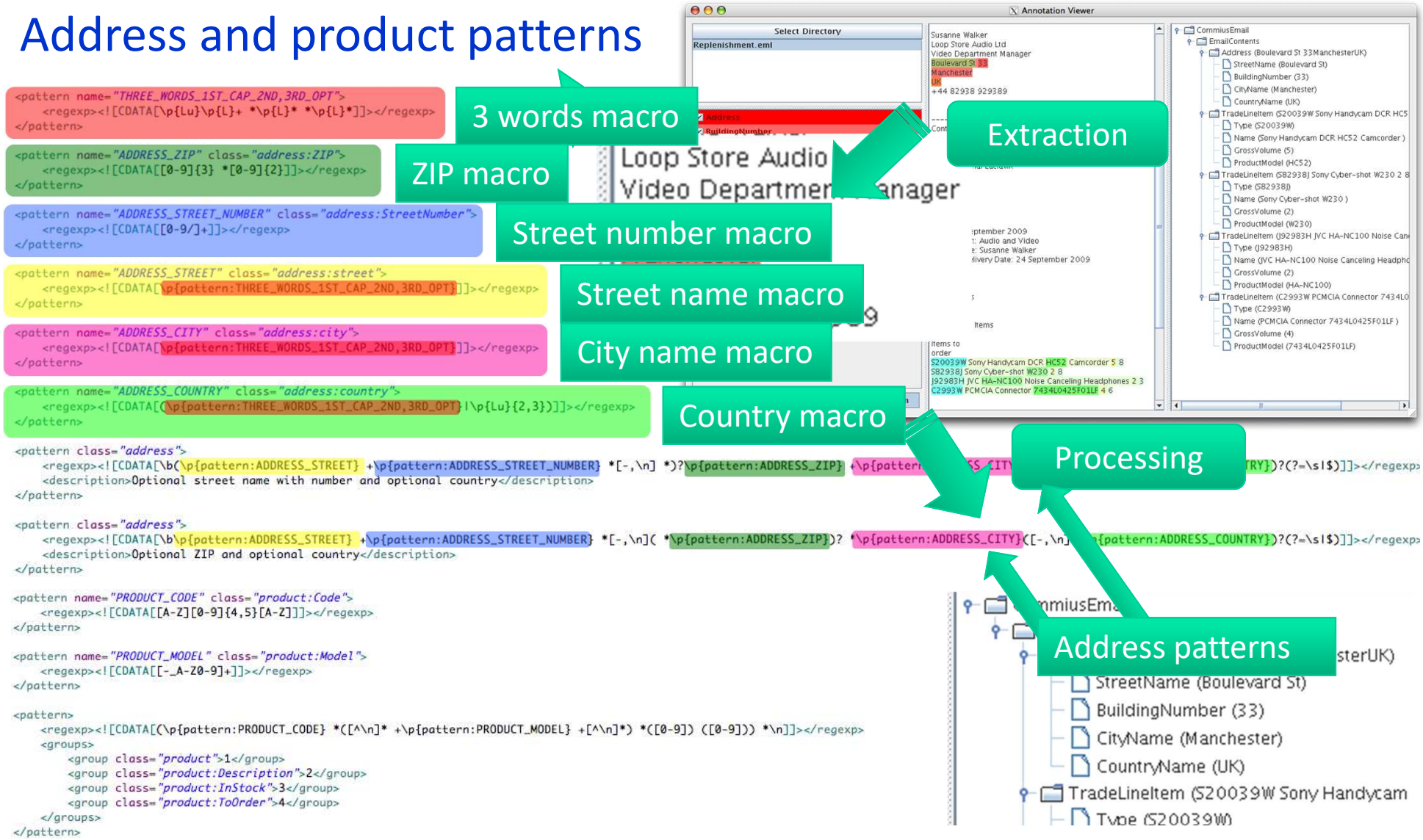
</patterns>
```



- Extraction patterns are defined in **XML** (we have XSD Schema)
- Macros can be used unlimited times in any pattern
- Macros in macros (any level)
- **Results** can be enhanced by GATE annotations (e.g. gazetteer lookups)
- Macros could be used to create new patterns only by clicking

Ontea Extraction Model

Address and product patterns




```
<pattern name="THREE_WORDS_1ST_CAP_2ND,3RD_OPT">
    <regexp><![CDATA[\p{Lu}\p{L}+ *\p{L}* *\p{L}*]]></regexp>
</pattern>

<pattern name="ADDRESS_ZIP" class="address:ZIP">
    <regexp><![CDATA[[0-9]{3} *[0-9]{2}]]></regexp>
</pattern>

<pattern name="ADDRESS_STREET_NUMBER" class="address:StreetNumber">
    <regexp><![CDATA[[0-9/]+]]></regexp>
</pattern>

<pattern name="ADDRESS_STREET" class="address:street">
    <regexp><![CDATA[\p{pattern:THREE_WORDS_1ST_CAP_2ND,3RD_OPT}]]></regexp>
</pattern>

<pattern name="ADDRESS_CITY" class="address:city">
    <regexp><![CDATA[\p{pattern:THREE_WORDS_1ST_CAP_2ND,3RD_OPT}]]></regexp>
</pattern>

<pattern name="ADDRESS_COUNTRY" class="address:country">
    <regexp><![CDATA[(\p{pattern:THREE_WORDS_1ST_CAP_2ND,3RD_OPT}|\p{Lu}{2,3})]]></regexp>
</pattern>

<pattern class="address">
    <regexp><![CDATA[\b(\p{pattern:ADDRESS_STREET} +\p{pattern:ADDRESS_STREET_NUMBER} *[-,\n] *)?\p{pattern:ADDRESS_ZIP} +\p{pattern:ADDRESS_CITY}([-,\n] *\p{pattern:ADDRESS_COUNTRY})?(?=\s!$)]]></regexp>
    <description>Optional street name with number and optional country</description>
</pattern>

<pattern class="address">
    <regexp><![CDATA[\b\p{pattern:ADDRESS_STREET} +\p{pattern:ADDRESS_STREET_NUMBER} *[-,\n]( *\p{pattern:ADDRESS_ZIP})? *\p{pattern:ADDRESS_CITY}([-,\n] *\p{pattern:ADDRESS_COUNTRY})?(?=\s!$)]]></regexp>
    <description>Optional ZIP and optional country</description>
</pattern>

<pattern name="PRODUCT_CODE" class="product:Code">
    <regexp><![CDATA[[A-Z][0-9]{4,5}[A-Z]]]></regexp>
</pattern>

<pattern name="PRODUCT_MODEL" class="product:Model">
    <regexp><![CDATA[[-_A-Z0-9]+]]></regexp>
</pattern>

<pattern>
    <regexp><![CDATA[(\p{pattern:PRODUCT_CODE} *([^\n]* +\p{pattern:PRODUCT_MODEL} +[^\n]*) *([0-9]) ([0-9])) *\n]]></regexp>
    <groups>
        <group class="product">1</group>
        <group class="product:Description">2</group>
        <group class="product:InStock">3</group>
        <group class="product:ToOrder">4</group>
    </groups>
</pattern>
```

Skúška

- email
- URL
- Číslo (peniaze)
- PSČ mesto
- Firma